# LAB 5: STATIC CODE ANALYSIS

Department of Computer Science and Engineering

Name: Parigna P V

SRN: PES2UG24CS909

SEM: 5TH G(CSE)

**1** Objective

To analyze the hostel room allocation Python program using Pylint, Bandit, and Flake8 to identify and fix code quality, security, and style issues.

**2** Tools Used

- Pylint — code quality analysis
- Bandit — security analysis
- Flake8 — Python style checking

3 Initial Ananlysis Results

```
(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>pylint hostel_allocation.py > pylint_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>bandit -r hostel_allocation.py > bandit_report.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.15

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>flake8 hostel_allocation.py > flake8_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad pylint_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad bandit_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad flake8_report.txt
```

Flake 8 :

```
hostel_allocation.py:1:1: F401 'os' imported but unused
hostel_allocation.py:2:1: F401 'sys' imported but unused
hostel_allocation.py:20:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:25:80: E501 line too long (92 > 79 characters)
hostel_allocation.py:29:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:30:8: E721 do not compare types, for exact checks use `is` / `is not`, for instance checks use
`isinstance()`
hostel_allocation.py:41:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:47:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:52:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:55:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:58:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:65:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:66:80: E501 line too long (81 > 79 characters)
hostel_allocation.py:68:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:71:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:74:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:79:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:82:1: E302 expected 2 blank lines, found 1
hostel_allocation.py:95:80: E501 line too long (116 > 79 characters)
hostel_allocation.py:101:1: E305 expected 2 blank lines after class or function definition, found 1
```

Bandit report :

Test results: >> Issue: [B506:yaml_load] Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load(). Severity: Medium Confidence: High CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html) More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b506_yaml_load.html Location: .\hostel_allocation.py:21:13 20 def loadConfig(text=CONFIG): 21 config = yaml.load(text, Loader=yaml.Loader) 22 for block in config["blocks"]: ---------- >> Issue:

[B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes. Severity: Low Confidence: High CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html) More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random Location: .\hostel_allocation.py:56:11 55 def roomLottery(student_ids): 56 return random.sample(student_ids, len(student_ids)) 57 -------------------------------------------------- >> Issue: [B106:hardcoded_password_funcarg] Possible hardcoded password: 'Hostel@123' Severity: Low Confidence: Medium CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html) More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b106_hardcoded_password_funcarg.html Location: .\hostel_allocation.py:66:11 65 def connectWardenDb(): 66 return sqlite_connect(host="localhost", user="warden", password="Hostel@123") 67 -------------------------------------------------- >> Issue: [B612:logging_config_insecure_listen] Use of insecure logging.config.listen detected. Severity: Medium Confidence: High CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html) More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b612_logging_config_insecure_listen.html Location: .\hostel_allocation.py:72:11 71 def startRemoteLogConfig(): 72 return logging.config.listen(9999) 73 -------------------------------------------------- >> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code. Severity: High Confidence: Medium CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html) More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201_flask_debug_true.html Location: .\hostel_allocation.py:80:4 79 def runPortal(): 80 app.run(debug=True) 81 -------------------------------------------------- Code scanned: Total lines of code: 84 Total lines skipped (#nosec): 0 Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0 Run metrics: Total issues (by severity): Undefined: 0 Low: 2 Medium: 2 High: 1 Total issues (by confidence): Undefined: 0 Low: 0 Medium: 2 High: 3 Files skipped (0):

Pylint report :

********** Module hostel_allocation hostel_allocation.py:95:0: C0301: Line too long (116/100) (line-too-long) hostel_allocation.py:1:0: C0114: Missing module docstring (missing-module-docstring) hostel_allocation.py:6:0: E0401: Unable to import 'flask' (import-error) hostel_allocation.py:20:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:20:0: C0103: Function name "loadConfig" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:29:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:29:0: C0103: Function name "allocateRoom" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:29:0: W0102: Dangerous default value [] as argument (dangerous-default-value) hostel_allocation.py:30:7: C0123: Use isinstance() rather than type() for a typecheck. (unidiomatic-typecheck) hostel_allocation.py:29:35: W0613: Unused argument 'gender' (unused-argument) hostel_allocation.py:41:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:41:0: C0103: Function name "vacateRoom" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:47:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:47:0: C0103: Function name "monthlyFee" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:52:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:52:0: C0103: Function name "priorityOrder" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:55:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:55:0: C0103: Function name "roomLottery" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:58:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:58:0: C0103: Function name "vacantBeds" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:60:4: C0206: Consider iterating with .items() (consider-using-dict-items) hostel_allocation.py:65:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:65:0: C0103: Function name "connectWardenDb" doesn't conform to snake_case naming style

(invalid-name) hostel_allocation.py:68:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:71:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:71:0: C0103: Function name "startRemoteLogConfig" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:75:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:75:0: C0103: Function name "roomLookup" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:79:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:79:0: C0103: Function name "runPortal" doesn't conform to snake_case naming style (invalid-name) hostel_allocation.py:82:0: C0116: Missing function or method docstring (missing-function-docstring) hostel_allocation.py:1:0: W0611: Unused import os (unused-import) hostel_allocation.py:2:0: W0611: Unused import sys (unused-import) ---------- Your code has been rated at 5.19/10

4 Issue Documentation Table

| Issue | Type | Line(s) | Description | Fix Approach |
|---|---|---|---|---|
| Unsafe YAML loading | Security | 21 | yaml.load() with yaml.Loader may allow arbitrary object construction. | Replaced yaml.load() with yaml.safe_load(). |
| Flask debug | Security | 80 | debug=True can expose | Changed to debug=False. |

| | | | | |
|---|---|---|---|---|
| mode enabled | | | the interactive debugger and risk code execution. | |
| Hardcoded password | Security | 66 | Database password was directly written in the source code. | Read the password from the WARDEN_DB_PASSWORD environment variable. |
| Insecure remote logging listener | Security | 72 | Remote logging configuration can expose the application to unsafe configurations. | Removed the remote logging listener because it was not required. |
| Mutable default argument | Bug | 29 | preferred=[] uses a mutable default argument that can be shared across calls. | Changed the default to None and initialized the list inside the function. |
| Incorrect type checking | Code Quality | 30 | type(year) != int is not the recommended way to check types. | Used isinstance() and explicitly rejected Boolean values. |
| Unused imports | Code Quality | 1–2 | The original code imported os and sys without using them. | Removed unused imports; retained os for reading the environment variable. |
| Missing docstrings | Code Quality | Multiple | Functions and the module lacked descriptive | Added module and function docstrings. |

| | | | | |
|---|---|---|---|---|
| | | | documentation. | |
| Invalid function naming | Code Style | Multiple | Function names such as loadConfig() did not follow snake_case. | Renamed functions, for example, to load_config() and allocate_room(). |
| Long lines | Code Style | 25, 66, 95 | Some lines exceeded the configured maximum length. | Split long statements across multiple lines. |
| Incorrect blank-line spacing | Code Style | Multiple | Function definitions did not follow PEP 8 spacing conventions. | Corrected blank lines between top-level definitions. |
| Missing gender and capacity checks | Functional Bug | 29–38 | Original allocation logic did not enforce room gender restrictions or correctly prevent over-allocation. | Added gender matching and corrected the capacity check. |
| Incorrect room vacancy handling | Functional Bug | 41–45 | Vacating a room removed the allocation record but left the student in the room's occupant list. | Removed the student from both the allocation record and the occupant list. |
| Duplicate student allocation | Functional Bug | 29–38 | A student could be allocated more than once. | Added a check to prevent duplicate allocations. |

## **5** Corrected Source Code

```
"""Hostel room allocation system with safer defaults and validation."""

import html
import os
import secrets
import yaml
from flask import Flask, request

# Global hostel state
rooms = {}
allocations = {}
waitlist = []
app = Flask(__name__)

CONFIG = """
blocks:
  A: {gender: M, rooms: 3, capacity: 2, fee: 4000}
  B: {gender: F, rooms: 2, capacity: 3, fee: 3500}
"""



def load_config(text=CONFIG):
    """Load hostel configuration safely and initialize rooms."""
    config = yaml.safe_load(text)
    if not isinstance(config, dict) or not isinstance(
        config.get("blocks"), dict
    ):
        raise ValueError("Configuration must contain a blocks mapping")

    rooms.clear()
```

```
  for block, info in config["blocks"].items():
    for number in range(1, info["rooms"] + 1):
      rooms[f"{block}{number}"] = {
        "gender": info["gender"],
        "capacity": info["capacity"],
        "fee": info["fee"],
        "occupants": [],
      }
  return config



def allocate_room(student_id, name, gender, year, preferred=None):
  """Allocate a student to a suitable room or add them to the waitlist."""
  if not isinstance(year, int) or isinstance(year, bool):
    return None
  if student_id in allocations:
    return allocations[student_id]["room"]

  preferred = preferred or []
  candidates = list(dict.fromkeys(preferred + list(rooms.keys())))

  for room_no in candidates:
    room = rooms.get(room_no)
    if room is None or room["gender"] != gender:
      continue
    if len(room["occupants"]) >= room["capacity"]:
      continue

    room["occupants"].append(student_id)
    allocations[student_id] = {
      "name": name,
      "room": room_no,
      "year": year,
    }
```

```
            if student_id in waitlist:
                waitlist.remove(student_id)
            return room_no

        if student_id not in waitlist:
            waitlist.append(student_id)
        return None


def vacate_room(student_id):
    """Remove a student from their room and allocation record."""
    allocation = allocations.pop(student_id, None)
    if allocation is None:
        return False

    room = rooms.get(allocation["room"])
    if room and student_id in room["occupants"]:
        room["occupants"].remove(student_id)
    return True


def monthly_fee(student_id, days_stayed):
    """Calculate the fee for completed 30-day periods."""
    if student_id not in allocations:
        raise ValueError("Student has no room allocation")
    if not isinstance(days_stayed, int) or days_stayed < 0:
        raise ValueError("Days stayed must be a non-negative integer")

    room = rooms[allocations[student_id]["room"]]
    return (days_stayed // 30) * room["fee"]


def priority_order(students):
    """Sort students by year, with senior students first."""
```

```
    return sorted(students, key=lambda student: student["year"], reverse=True)



def room_lottery(student_ids):
    """Randomize student order for a room lottery."""
    return secrets.SystemRandom().sample(student_ids, len(student_ids))



def vacant_beds():
    """Count beds in rooms that currently have no occupants."""
    return sum(
        room["capacity"]
        for room in rooms.values()
        if not room["occupants"]
    )



def connect_warden_db():
    """Read database credentials from environment variables."""
    password = os.environ.get("WARDEN_DB_PASSWORD")
    if not password:
        raise RuntimeError("Set the WARDEN_DB_PASSWORD environment variable")
    return sqlite_connect(
        host="localhost",
        user="warden",
        password=password,
    )



def sqlite_connect(host, user, password):
    """Simulate a database connection result for this lab."""
    return {"host": host, "user": user, "connected": bool(password)}
```

```
@app.route("/room")
def room_lookup():
    """Return the room allocation for the requested student ID."""
    student_id = request.args.get("id", "")
    safe_id = html.escape(student_id)
    allocation = html.escape(str(allocations.get(student_id)))
    return f"<h1>Room for {safe_id}: {allocation}</h1>"



def run_portal():
    """Run the Flask application without debug mode."""
    app.run(debug=False)



def main():
    """Demonstrate the hostel allocation functions."""
    load_config()
    print("Rooms:", list(rooms.keys()))
    print("S1 ->", allocate_room("S1", "Arjun", "M", 1))
    print("S2 ->", allocate_room("S2", "Kabir", "M", 2))
    print("S3 ->", allocate_room("S3", "Rohan", "M", 3))
    print("S4 (female) ->", allocate_room("S4", "Diya", "F", 1))
    print("S1 again ->", allocate_room("S1", "Arjun", "M", 1))
    print("S5 year '2' ->", allocate_room("S5", "Vikram", "M", "2"))
    print("Occupants of A1:", rooms["A1"]["occupants"])
    vacate_room("S2")
    print("Occupants of A1 after S2 left:", rooms["A1"]["occupants"])
    print("Fee for S1, 45 days:", monthly_fee("S1", 45))
    students = [{"year": 1}, {"year": 4}, {"year": 2}]
    print("Priority (seniors first?):", [
        student["year"] for student in priority_order(students)
    ])
    print("Lottery:", room_lottery(["S6", "S7", "S8"]))
    print("Vacant beds:", vacant_beds())
```

```
    try:
        print("Warden DB:", connect_warden_db())
    except RuntimeError as error:
        print("Warden DB:", error)
    print("Waitlist:", waitlist)


if __name__ == "__main__":
    main()
```

## 6 Final Analysis Results

```
(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>pylint hostel_allocation.py > pylint_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>bandit -r hostel_allocation.py > bandit_report.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.15

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>flake8 hostel_allocation.py > flake8_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad pylint_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad bandit_report.txt

(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>notepad flake8_report.txt
```

Flak8 report was empty hence no style violations were reported

Pylint report

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 5.19/10, +4.81)

```

bandit report

```
Run started:2026-10-19 08:57:12.574635+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 138
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

## **7** Program Execution

```
(venv) C:\Users\parig\OneDrive\Desktop\L5_60_hostel-room-allocat
ion>python hostel_allocation.py
Rooms: ['A1', 'A2', 'A3', 'B1', 'B2']
S1 -> A1
S2 -> A1
S3 -> A2
S4 (female) -> B1
S1 again -> A1
S5 year '2' -> None
Occupants of A1: ['S1', 'S2']
Occupants of A1 after S2 left: ['S1']
Fee for S1, 45 days: 4000
Priority (seniors first?): [4, 2, 1]
Lottery: ['S8', 'S6', 'S7']
Vacant beds: 5
Warden DB: Set the WARDEN_DB_PASSWORD environment variable
Waitlist: []
```

8 Reflection Questions

**1. Which issues were the easiest to fix, and which were the hardest? Why?**

The easiest issues to fix were unused imports, missing docstrings, naming convention violations, and formatting errors because they required straightforward changes. The harder issues involved unsafe YAML loading, hardcoded credentials, Flask debug mode, and room allocation logic because they required understanding security risks and the program's behavior.

**2. Did the static analysis tools report any false positives? If so, describe one example.**

Pylint reported that Flask could not be imported because it was not available in the active virtual environment. This was an environment configuration issue rather than necessarily a defect in the source code. Installing Flask in the virtual environment resolved the import issue.

**3. How would you integrate static analysis tools into your actual software development workflow?**

I would run Pylint, Bandit, and Flake8 during development and before committing code. I would also integrate them into a continuous integration

pipeline so that code quality, security issues, and style violations are checked automatically before merging changes.

**4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?**

The code became more readable through consistent naming conventions, docstrings, and formatting. Security improved by using safe YAML loading, disabling Flask debug mode, and removing hardcoded credentials. The allocation logic also became more robust through improved input validation, gender matching, capacity checks, duplicate prevention, and correct room vacancy handling. Pylint's score improved from 5.19/10 to 10.00/10, Bandit reported no issues, and Flake8 produced empty output.